

# 성과와 보안을 수치로 증명하는, Electron 데스크톱 앱 개발자 임세훈입니다.

[sakills914@gmail.com](mailto:sakills914@gmail.com) / GitHub : [Ummsehun](#)

경북소프트웨어마이스터고 게임개발과 재학 중



## 끊임없이 개선합니다.

한 번 만들고 끝내지 않습니다. JSON 저장 구조를 **SQLite**로 전환하는 등, 발견한 문제를 구조적으로 개선해 **실제 배포**까지 연결합니다.

## 단순 구현에서 멈추지 않고 디테일까지 챙깁니다.

"동작하는" 수준에 만족하지 않습니다. 벤치마크로 성능을 **수치화**하고, 사용자 경험까지 검증하여 **디테일**을 놓치지 않고 완성합니다.

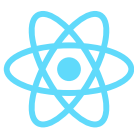
## 주요 기술



TypeScript



Electron



React



Tailwind CSS

## 주요 활동

2024.04~2024.08 : OTP 기획 동아리

- 프로젝트의 일정과 기능 범위를 정리 및 문서화하여 일정 조율

2024.08~2025.03 : AdA 실무 동아리

- API·데이터베이스 연동 등 백엔드 작업 경험

2026.03~ 현재 : Appsolute 앱 개발동아리

- PC 앱 개발 프로젝트 수행 (TermPlay 등)

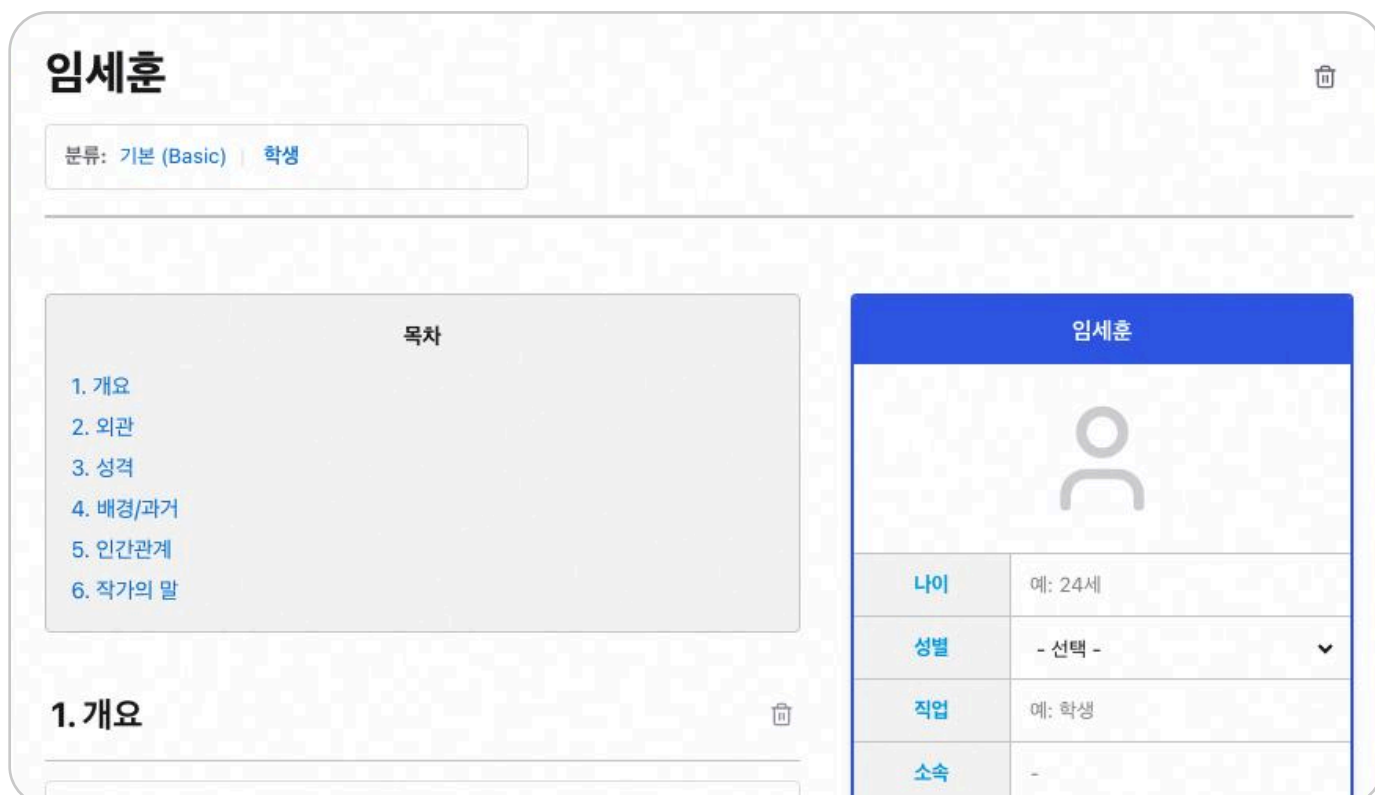
## Link

<https://github.com/ummsehun>

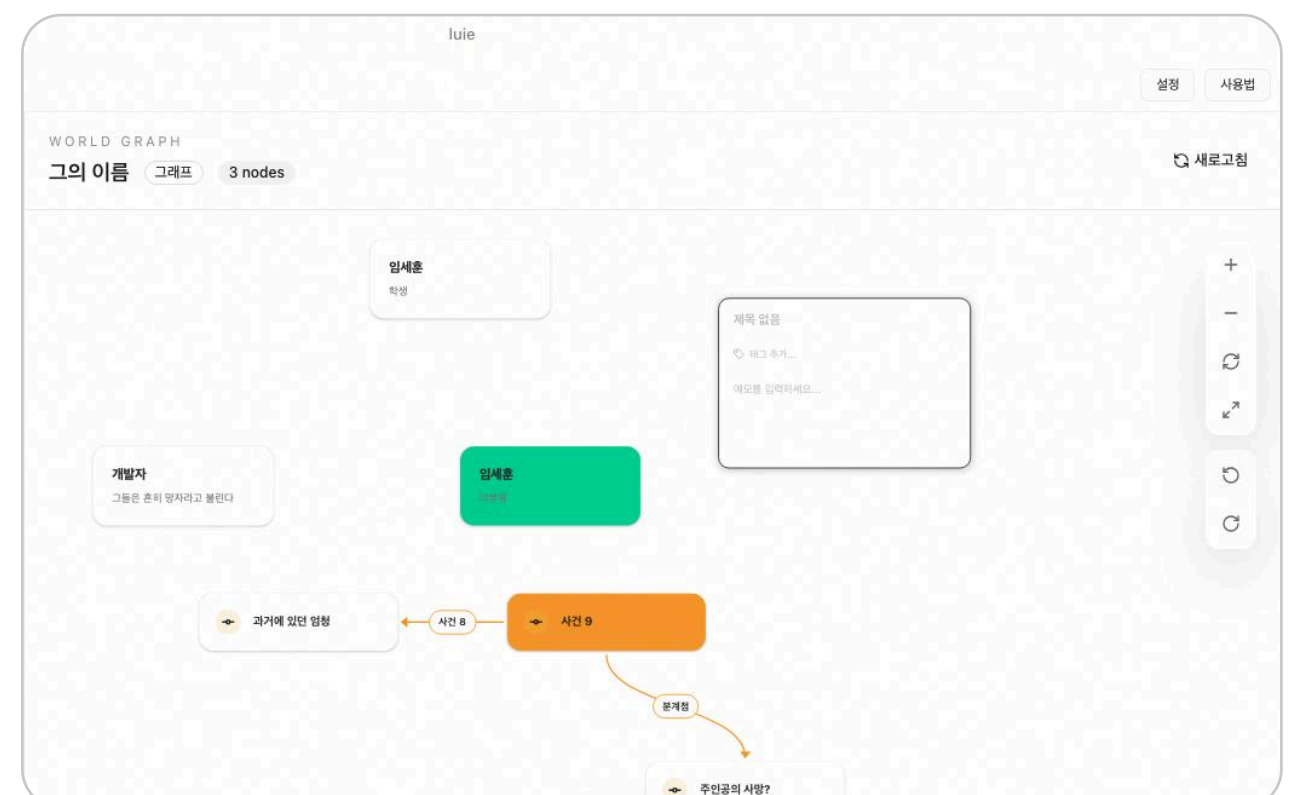


# Luie - 웹소설 작가들의 흐름을 구축하는 워드프로세서

Luie는 웹소설 작가의 집필·정리·관리를 하나의 흐름으로 통합한 **PC 앱** 워드프로세서입니다.  
사이드바·에디터·바인더 구조를 기반으로 직관적인 집필 환경을 구현했습니다.



- 등장인물 UI 구축



- React Flow를 사용하여 캔버스 UI 구축

## 서비스

[GitHub README](#)

[랜딩 페이지](#)

## 작업 기간

2026.1.15 ~ 2026.3.19

## 팀 프로젝트 : 2인 / 개발 단독

- 웹소설 특화 에디터 구현 : **Tiptap** 기반 원고 작성 에디터, 챕터 단위 편집 흐름, 툴바 및 작성 UX 구성
- Canvas / Graph View 구축 : **React Flow** 기반 인물·사건·관계 시각화 화면 구현 및 **노드/엣지 인터랙션** 설계
- 멀티 패널 레이아웃 시스템 구현 : **react-resizable-panels** 기반 구조와 패널 크기·열림 상태 유지 로직 개발
- 로컬 프로젝트 패키징 구조 구현 : **`.luie`** 파일 기반 저장·복구 흐름 설계 및 프로젝트 데이터 import/export UX 구성

## Luie 기술 스택



react-resizable-panels : 바인더, 에디터, 사이드 패널을 나누고 사용자의 작업 레이아웃 상태를 유지하도록 구현했습니다.



Electron: Renderer와 Main Process를 분리하고 로컬 파일 저장, 복구, import/export 기능을 IPC 흐름으로 연결했습니다.



SQLite : JSON ZIP 저장 구조를 SQLite 기반 .luie 포맷으로 전환해 부분 조회, 수정, 복구 성능을 개선했습니다.



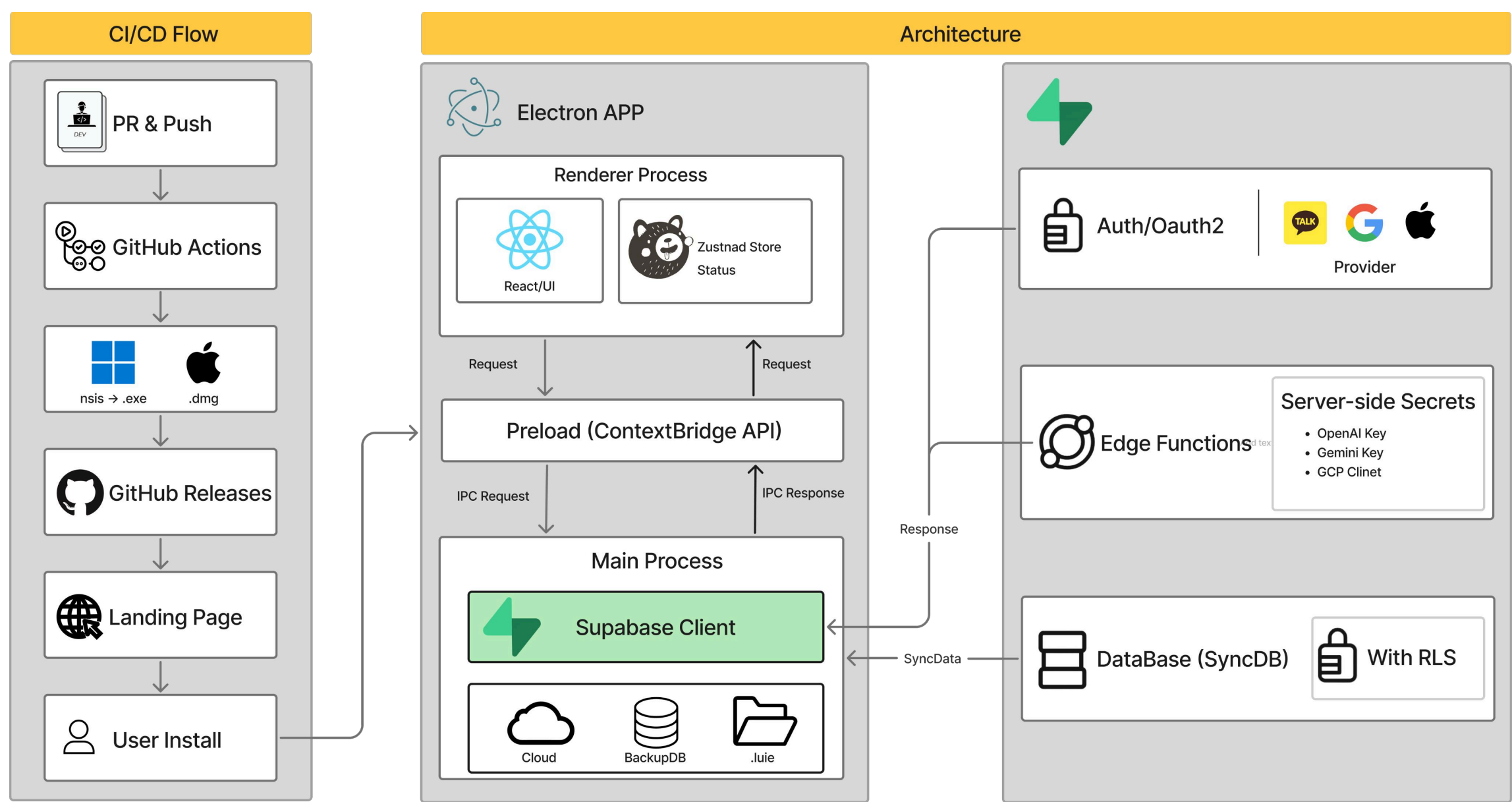
React Flow : 등장인물, 사건, 설정 간 관계를 노드와 엣지로 시각화하는 Graph View를 구성했습니다.



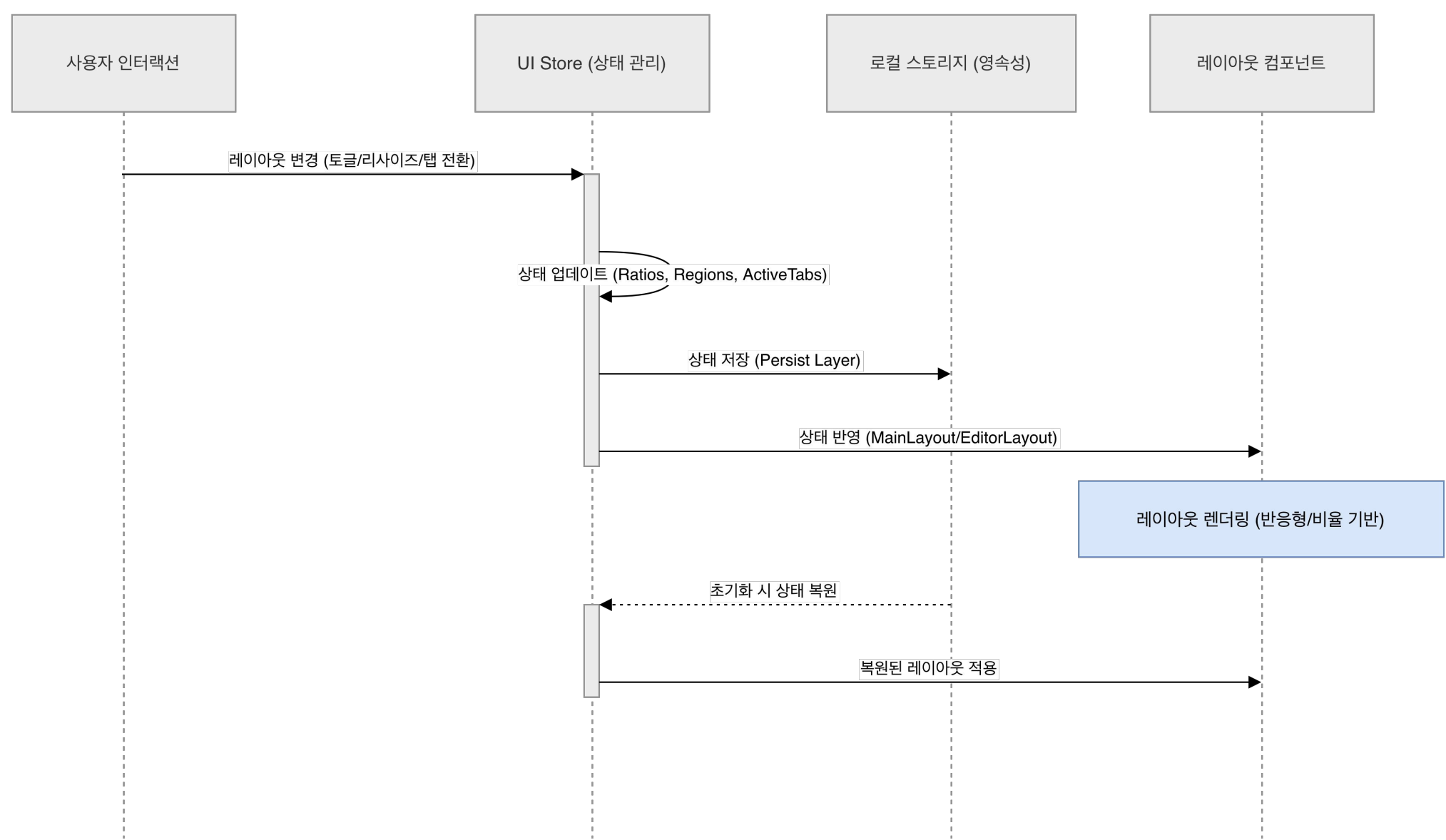
Tiptap : 웹소설 원고 작성에 맞춘 챕터 단위 에디터와 블록 기반 편집 UX를 구현했습니다.

# CI/CD 파이프라인 및 시스템 아키텍처 구성

## • 아키텍처



## • .luie 저장 파이프라인

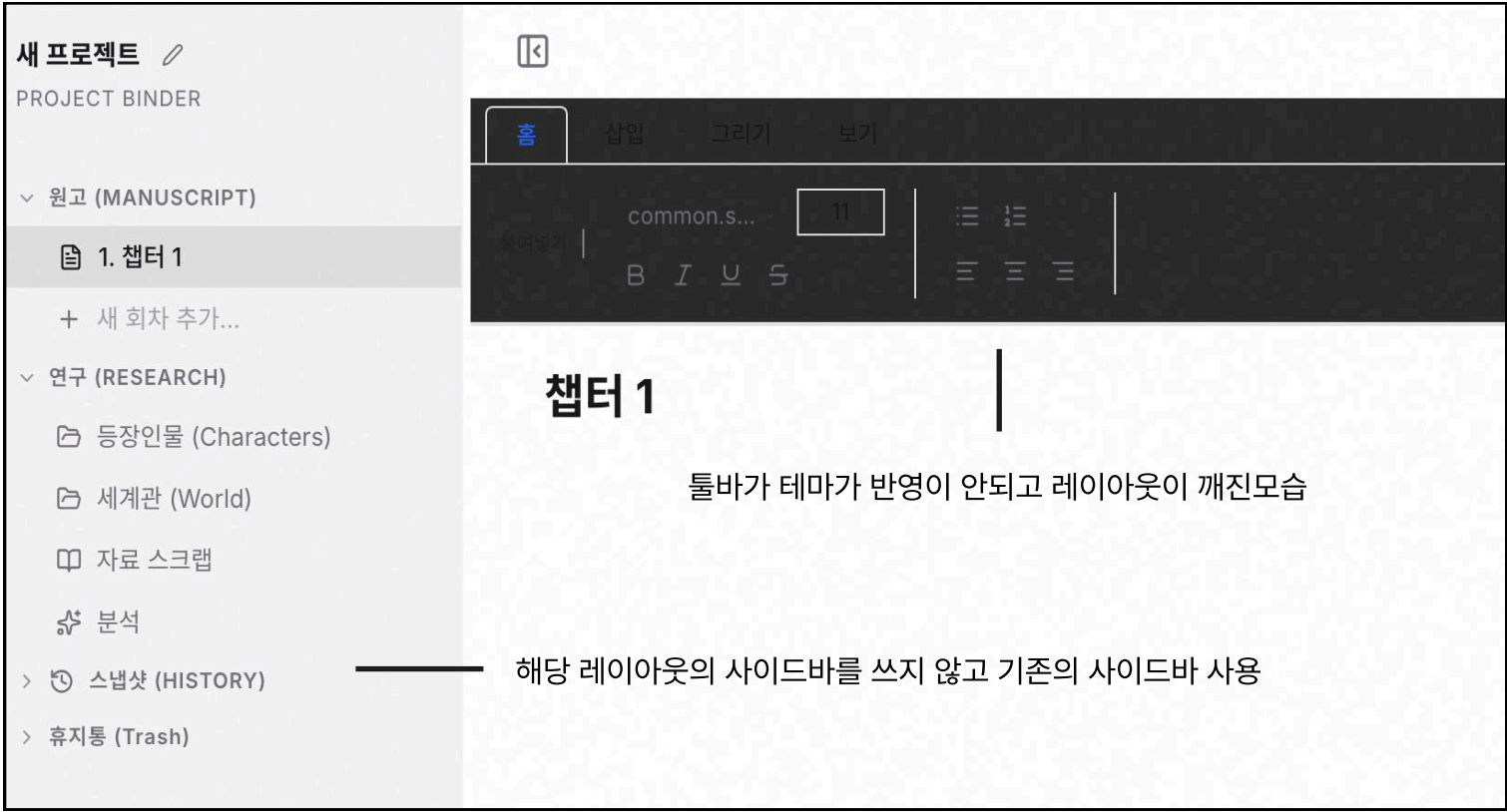


## • 아키텍처 설명

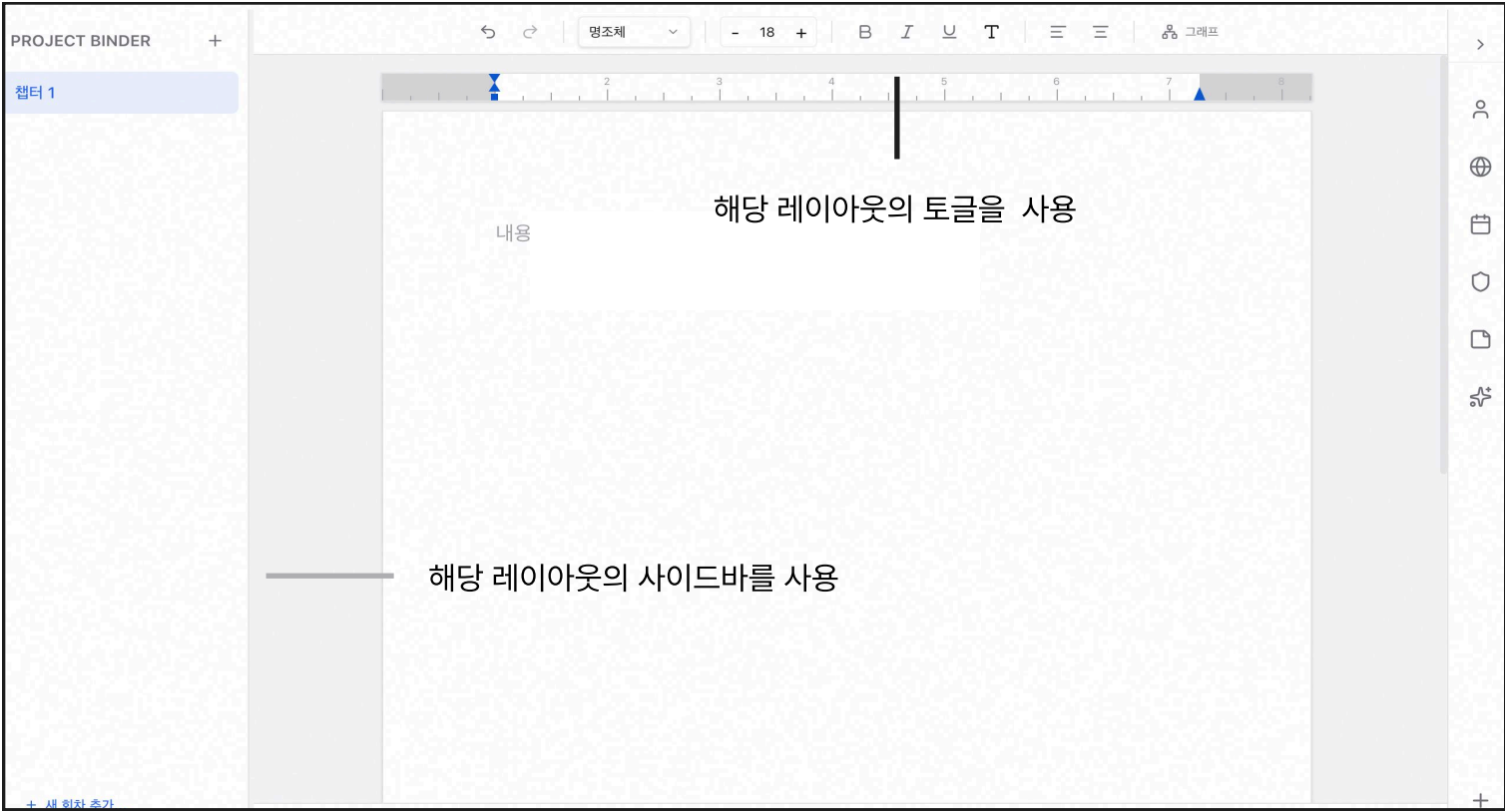
- Electron Main / renderer Process를 분리하여 IPC 기반 통신 구조로 로컬 파일 접근과 UI 영역을 분리
- GitHub Actions와 GitHub Release API를 활용해 태그 기반 배포 및 최신 버전 다운로드 흐름 구성
- renderer를 React, Tailwind CSS, Zustand를 활용해 화면 구성과 레이아웃 상태를 관리
- Main Process를 Cloud, Backup DB, **SQLite** 기반 '.luie' 문서 포맷과 연동



# 레이아웃 모드 오작동 문제



• 수정 전



• 수정 후

작가별 작업 환경을 지원하기 위해 3개의 추가 레이아웃을 구성하였지만 px 기반 **단일 상태**로인하여, 사이드바 토클이나 패널 변경 시 레이아웃이 서로 **겹치거나 깨지는** 문제가 발생하였습니다.

이를 해결하기 위해 **Zustand** 기반 영역별 상태 분리와 % 단위 레이아웃 구조를 적용해 레이아웃 충돌을 줄이고, 사이드바 토클 시 발생하던 레이아웃 깨짐을 해결하여 패널 간 자연스러운 리사이즈 및 **반응형 대응**이 가능해졌습니다.

## .luie 패키징 문제

기존 .luie는 **JSON** 기반 구조라 **읽기 속도**와 저장 **안정성**이 충분히 검증되지 않았습니다  
이러한 문제들을 해결하기 위해 다음과 같이 **SQLite**로 전환했습니다.

- **JSON → SQLite로 전환**

JSON ZIP 기반 저장 구조를 SQLite로 전환하고 300챕터 규모에서 벤치마크를 수행했습니다

- 전체 데이터 조회: 55ms → 17ms (약 3배 개선)
- 엔트리 목록 조회: 76ms → 2.4ms (약 30배 개선)
- 단일 데이터 수정: 17ms → 4.7ms (약 3배 개선)

### 저장 안정성 검증

- 일반·fullprod writing loop 각 3/3회 성공, 파생 작업 실패 0건
- 대규모 drain 테스트(300챕터 / 900 저장 샘플): 저장 지연 p95 약 13ms
- 약 69초 후 검색·메모리·요약·임베딩 큐 모두 완전히 비워짐 (pending/running/failed 0)

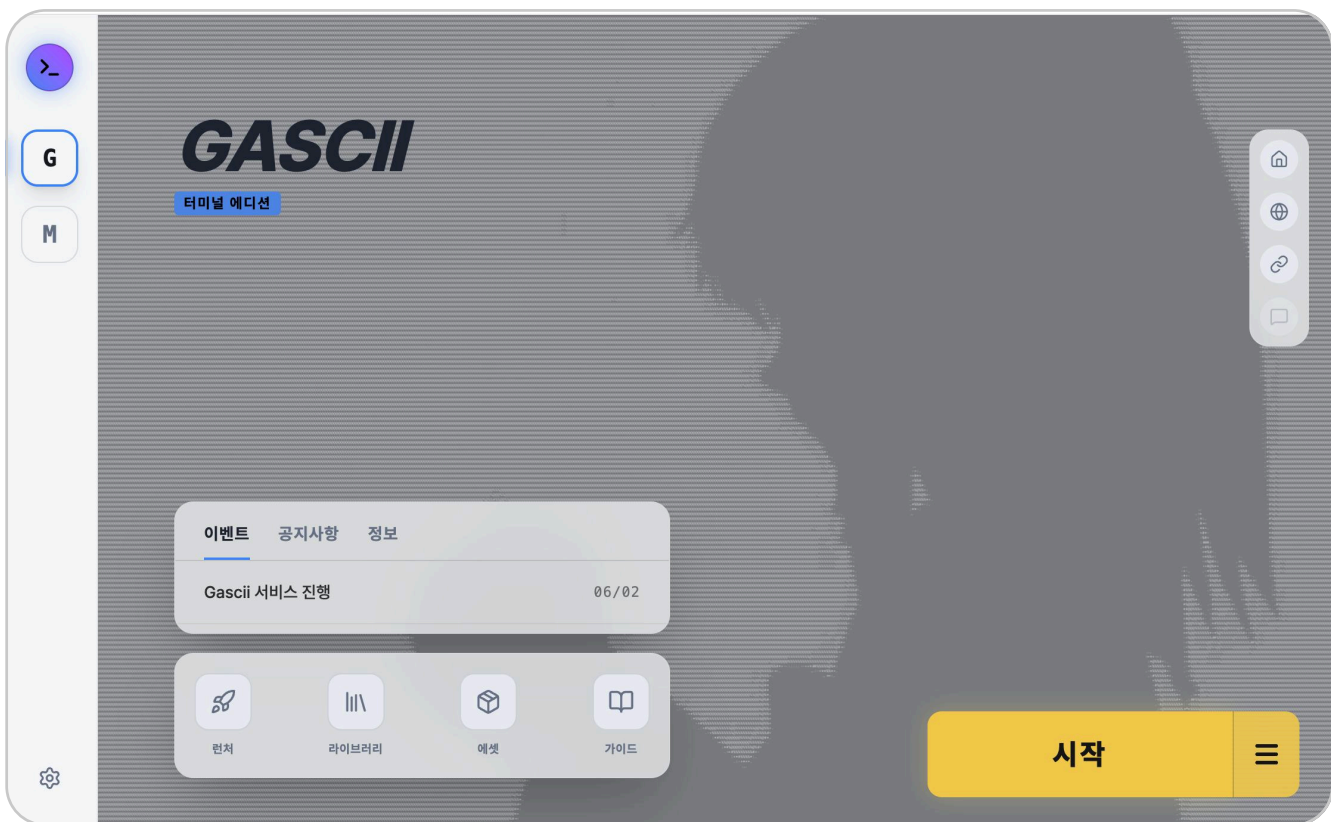
특히 데이터 규모가 증가할수록 부분 조회 및 수정 성능에서 SQLite의 구조적 이점이 확인되었으며, full production 모드에서 저장실패나 파생작업 없이 큐가 끝까지 정리되는 것을 확인했습니다.



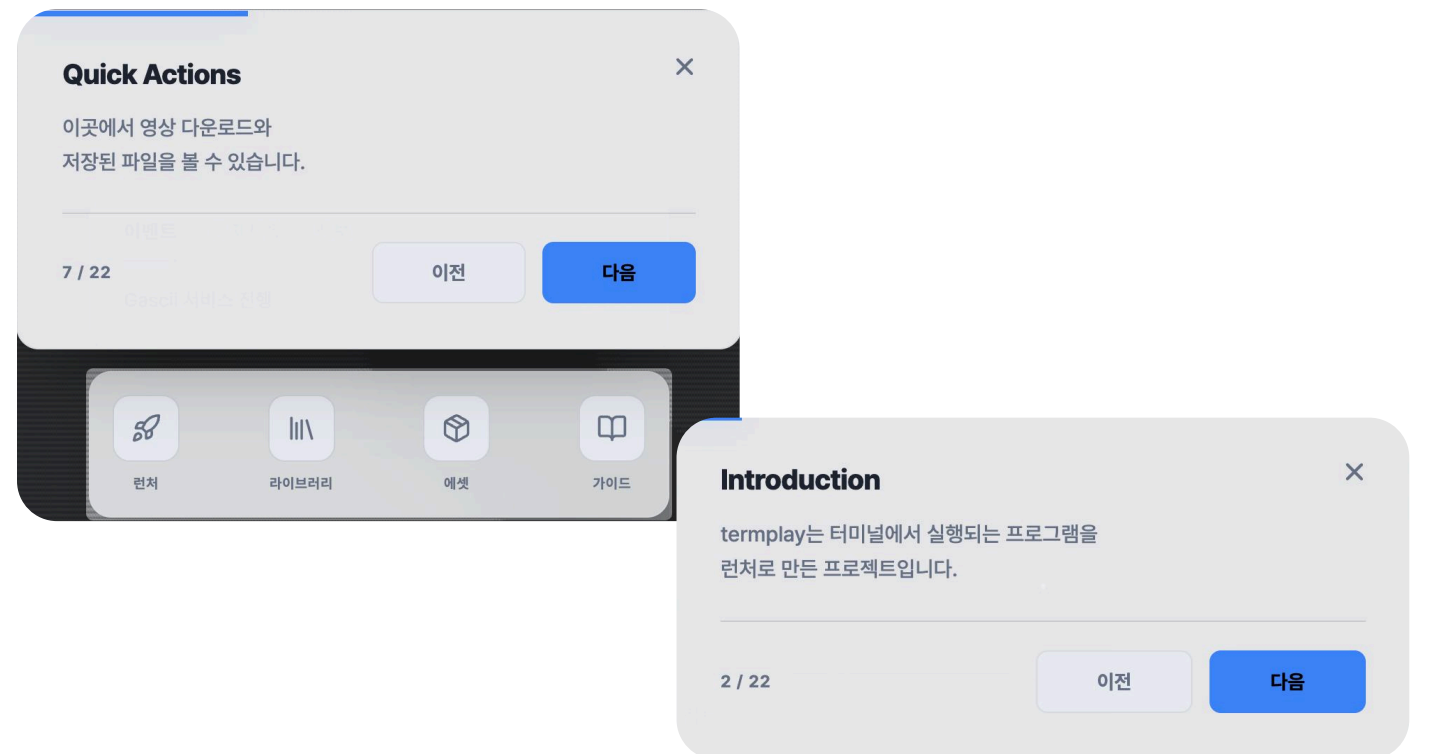


# TermPlay - 터미널 기반 콘텐츠를 실행까지 연결하는 런처

TermPlay는 터미널 기반 콘텐츠를 다운로드, 검증, 설치, 실행까지 연결하는 런처입니다.  
SHA-256 검증과 샌드박스 기반 실행 환경으로 보안을 강화했습니다.



- TermPlay의 Client UI



- driver.js를 사용하여 가이드 구성

## 서비스

[GitHub README](#)

[랜딩 페이지](#)

## 작업 기간

2026.4.3 ~ 2026.5.04

## 개인 프로젝트

- driver.js 기반 온보딩 가이드 구축 : 주요 런처 기능과 설정 흐름을 단계별로 안내하는 사용자 **투어 UX 구현**
- Electron 권한 분리 구조 설계 : main / preload / renderer 역할을 분리하고 **제한된 IPC API**로 네이티브 기능 연동
- SHA-256 무결성 검증 로직 구현 : release asset과 추가 에셋의 **checksum 비교**를 통해 손상·변조 파일 실행 방지
- 보안 실행 환경 구성 : CSP, **sandbox**, 외부 URL allowlist, Linux/macOS 샌드박스 정책을 적용해 **실행 경계 강화**

## TermPlay 기술 스택



Staging Install: 다운로드한 archive를 최종 설치 경로에 바로 풀지 않고 압축 안전성, 심볼릭 링크 여부를 검증한 뒤 반영했습니다.



GitHub Releases : 환경을 직접 준비하지 않아도 되도록 플랫폼별 사전 빌드된 바이너리를 내려받는 설치 흐름을 구성했습니다.



Electron IPC: renderer와 Main Process를 분리하고 다운로드, 설치, 검증, 실행 권한 경계를 분리했습니다.



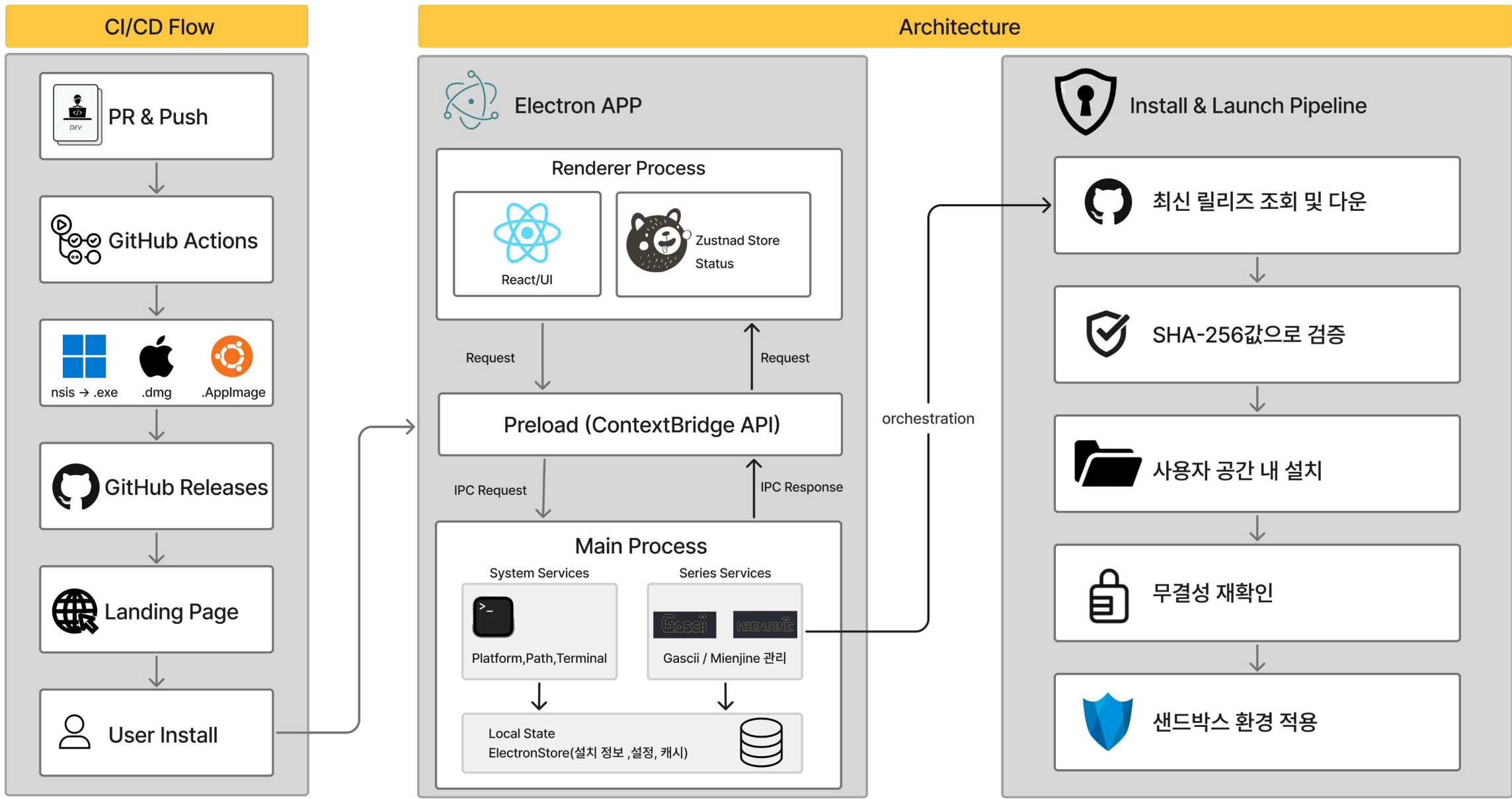
Sandbox: nodeIntegration 비활성화, contextIsolation, URL allowlist로 실행 권한을 제한했습니다.



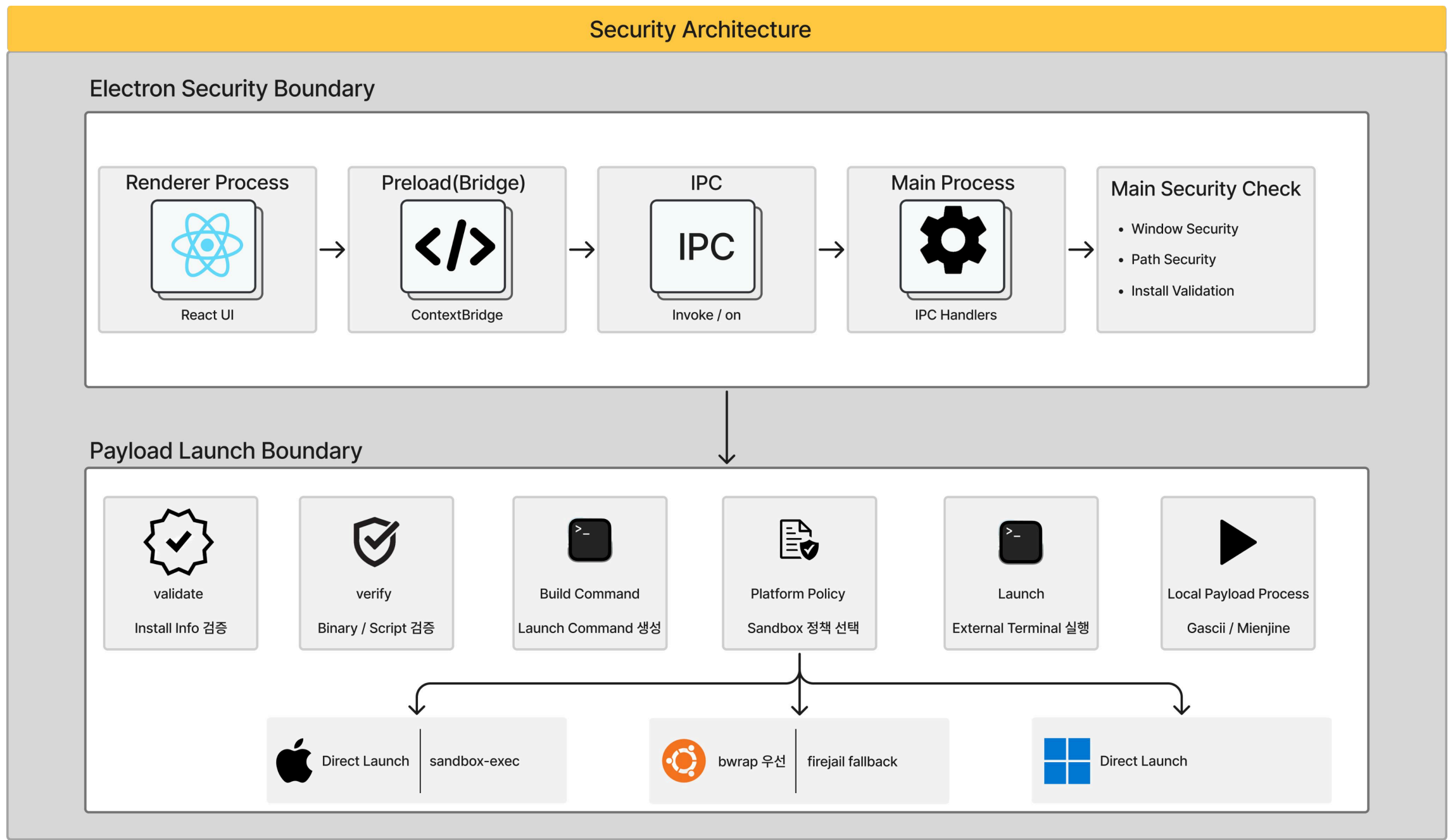
SHA-256 검증 : asset checksum을 검증해 손상·변조된 파일 실행을 차단했습니다.

# CI/CD 파이프라인 및 시스템 아키텍처 구성

## • 아키텍처



## • 보안 아키텍처



## • 아키텍처 설명

- Main / renderer / preload를 분리하고 IPC 구조를 적용하여 UI 영역과 로컬 파일, 프로세스 실행 권한 분리
- renderer는 Zustand Store로 상태를 관리하고, Main은 Series Services로 다운로드, 검증, 설치 담당 구성
- **GitHub Actions**와 Releases를 활용하여 Windows 등 배포 및 릴리스 조회 및 무결성 흐름 설계

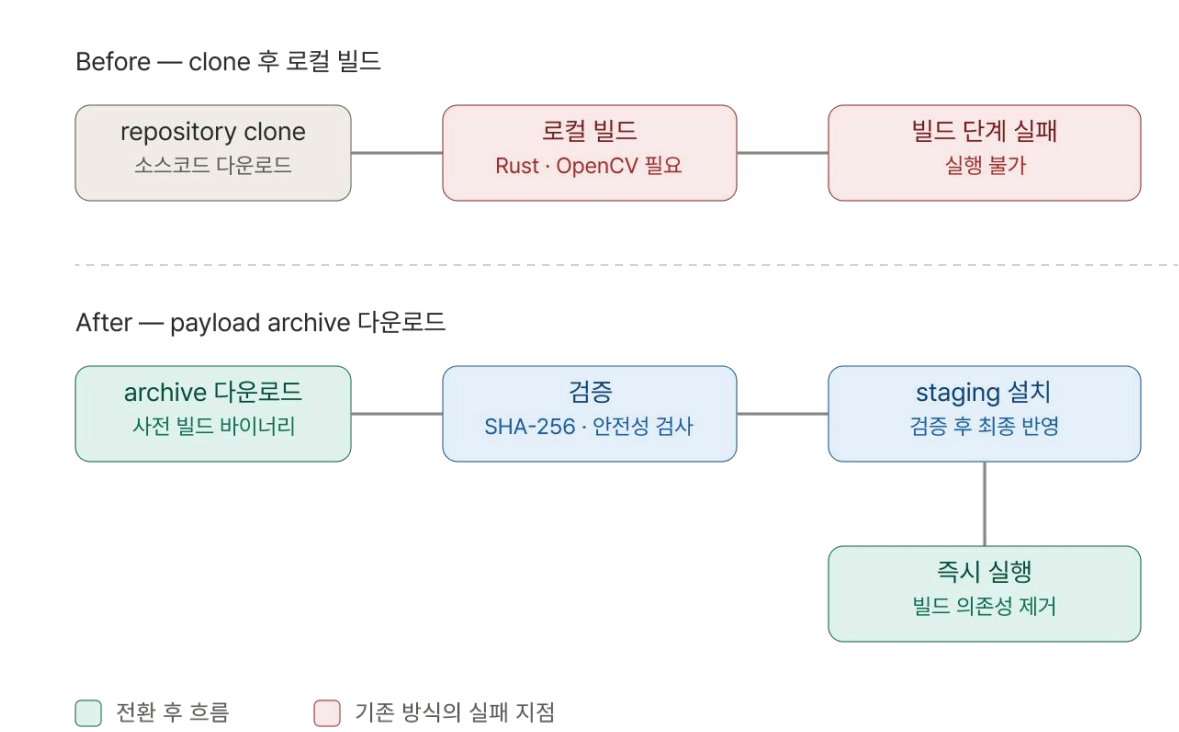
## 외부 터미널 프로그램 설치·실행 자동화

런처에서 Gascii, Mienjine 같은 터미널 프로그램을 실행하기 위해,  
처음에는 GitHub repository를 clone해 로컬에서 빌드하는 방식으로 설계했습니다.

그러나 이 방식은 사용자의 PC에 네이티브 빌드 의존성이 갖춰져 있어야 한다는 전제가 있었고,  
일반 사용자 환경에는 이런 의존성이 없어 Clone을 받아도 빌드 단계에서 실행이 불가능하였습니다.

- Clone → payload archive로 변경

사용자 환경에서 빌드하는 대신 플랫폼별로 바이너리를 payload archive로,  
다운로드하는 방식을 채택하였습니다 이로써 빌드 의존성을 제거하여 받는 즉시 실행이 가능케 구성하였습니다.



## 보안 검증 자동화

외부 **archive**를 받아 로컬에 풀고 실행하는 구조므로 무결성이 훼손되거나,  
압축해제 과정이 안전하지 않을 시 Electron 권한 경계가 약해지면 임의 파일 실행으로 이어질 수 있었습니다.

- 기존 **smoke test** 중심 구조 → 보안 경계별 테스트 스크립트 구조로 확장

기존 smoke test 중심 구조를 보안 경계별 테스트로 확장하여 다음을 자동 회귀 탐지하도록 했습니다.

- Electron 권한 경계 : contextIsolation / sandbox / nodeIntegration 회귀 방지  
raw ipcRenderer 노출 차단 , CSP unsafe-inline·unsafe-eval 차단
- 무결성 : SHA-256 digest mismatch / malformed hash 차단, staging 기반 설치 반영
- archive escape 차단 : tar.gz·zip path traversal, path entry, symlink 포함 archive 차단

이로써 외부 payload 설치·실행 흐름에서 무결성 훼손, archive escape  
보안 설정 약화를 자동으로 탐지할 수 있게 됐습니다.

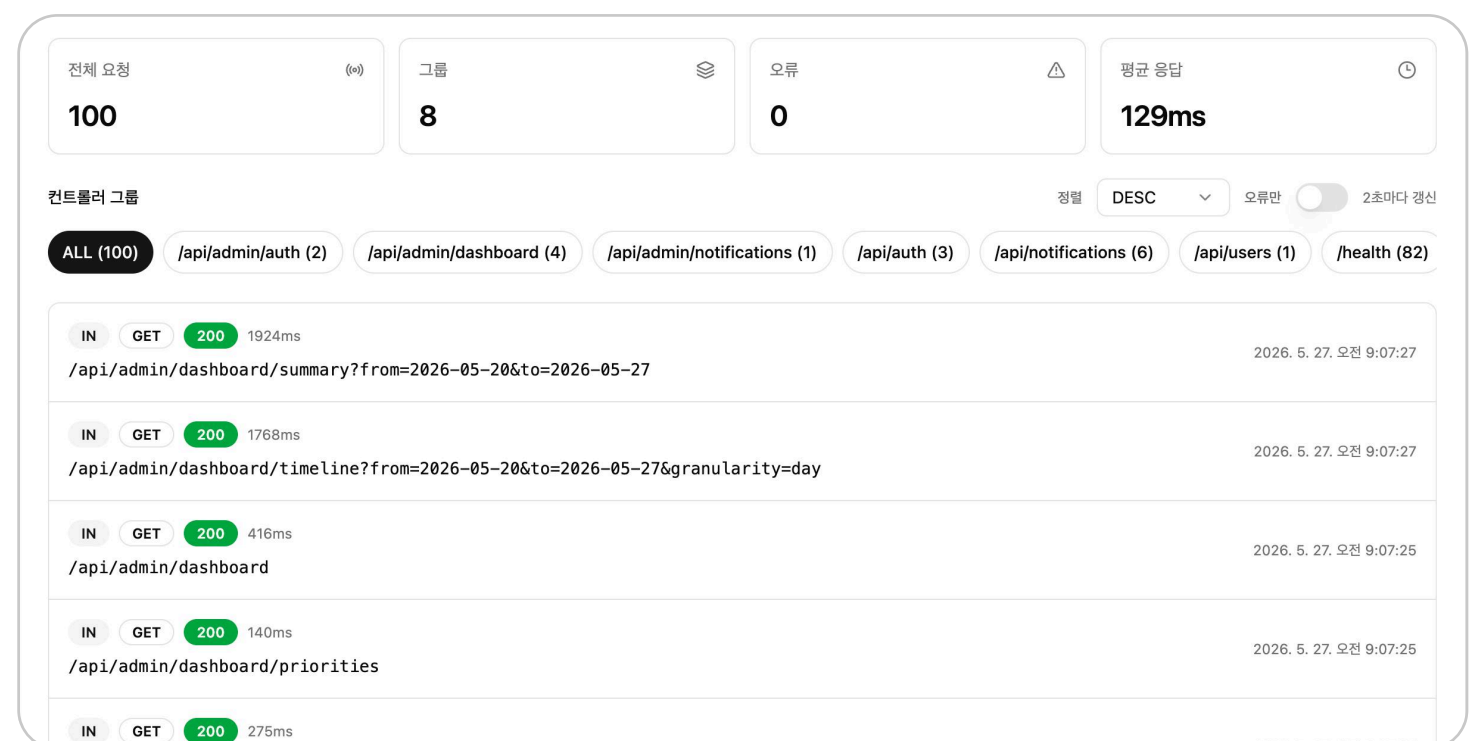
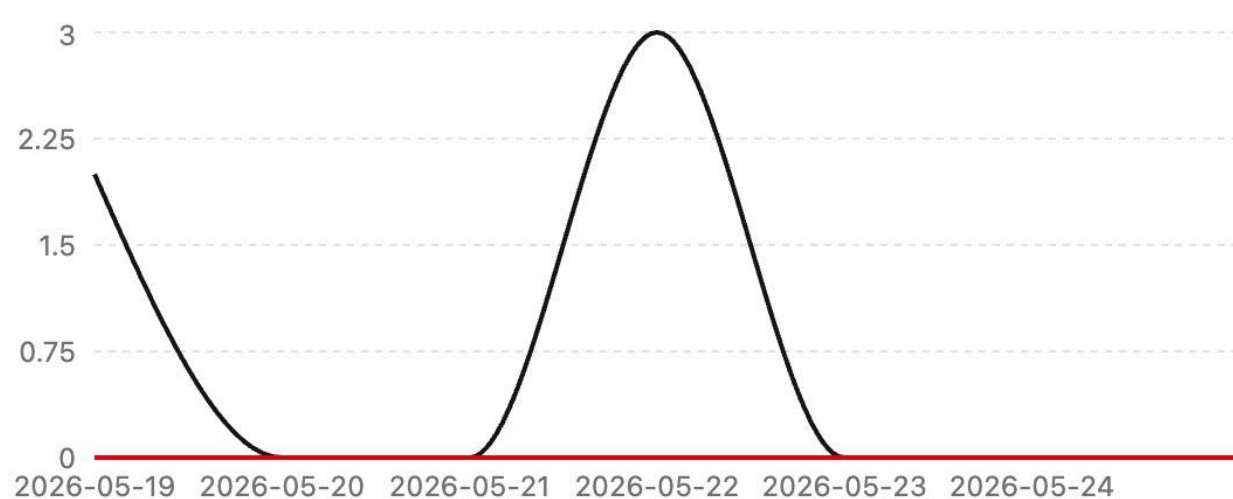




# Paw - 유기·실종 동물을 돕는 모바일 앱 운영 어드민 페이지

Paw는 유기·실종 동물 공고, 제보, 보호소 정보를 제공하는 반려동물 구조 모바일 앱입니다. 공고 승인, 서버 확인 등 운영 업무를 웹에서 처리하는 어드민 페이지를 구현했습니다.

신고/처리 타임라인



- Chart.js를 이용한 신고/처리 타임라인

- 실시간 서버 로그 조회

## 서비스

[GitHub README](#)

## 작업 기간

2026.3.4 ~ 2026.6.4

## 팀 프로젝트 : 5명 / 어드민 FE 전담

- 도메인 관리 UI : 사용자, 실종/유기 동물 공고, 커뮤니티, 알림 발송 시스템 **인터페이스 개발**
- Passkey (WebAuthn) 기반 인증 모듈 구현 : **비밀번호 없는** 보안 로그인 플로우 구축
- 데이터 시각화 : 대시보드 내 **실시간 운영 지표** 및 신고 타임라인 차트 배치

## Paw 기술 스택

 React : 공고, 커뮤니티, 알림 관리 화면 등 도메인 단위로 분리해 운영자가 필요한 기능에 접근할 수 있는 어드민 UI를 구성했습니다.

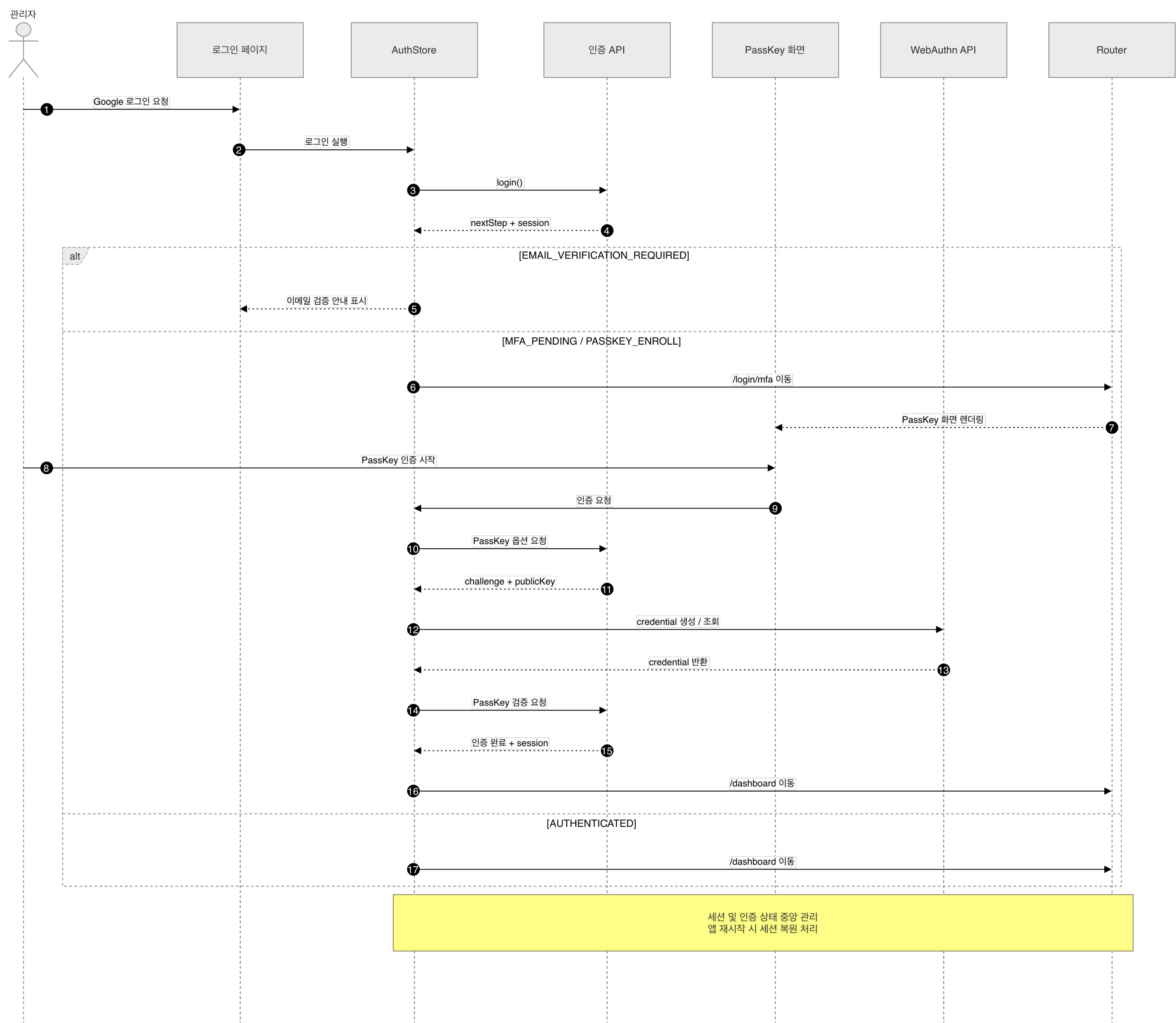
 Zustand : 관리자 세션, 인증 단계 등 상태를 전역 상태로 분리해 로그인 이후의 관리 흐름과 화면 상태를 일관되게 유지했습니다.

 TypeScript : API 응답 타입을 명시해 화면 간 데이터 계약을 안정화하여 타입 오류를 조기에 확인할 수 있게 했습니다.

 WebAuthn : 서버의 인증 단계 응답을 기준으로 로그인 흐름을 전환하여, Passkey 기반 로그인 구조를 구현했습니다.



# 패스키 시퀀스 다이어그램



## • 다이어그램 설명

- Router 기반 단계 전환 구조를 적용해 /login → /login/mfa → /dashboard 흐름을 명시적으로 관리
- WebAuthn 호출을 Passkey 화면 계층으로 분리하여 브라우저 인증 처리와 UI 책임을 명확히 분리
- AuthStore에서 세션과 인증 상태를 관리하여 라우팅 전환 후에도 Passkey 진행 상태가 유지
- 상태를 분리하여 기존 사용자 인증 검증과 신규 Passkey 등록 흐름을 독립적으로 처리
- 서버가 반환하는 nextStep 기반 상태 머신 구조로 인증 단계를 제어

# 어드민 UI 디자인 디테일

## Dashboard 개요

서비스 운영 현황을 한 화면에서 빠르게 파악할 수 있도록 핵심 지표 중심으로 구성했습니다.

대시보드 개요

실시간 운영 지표를 모니터링합니다.

현재 접속중인 사용자

0

서버 상태

OPERATIONAL

오늘 신고

0

대기 신고

0

선택 기간 운영 지표

회원가입

10

신규 공고

31

커뮤니티 글

2

처리된 작업

0

접수된 신고

0

숨김 콘텐츠

0

제재 사용자

0

신고/처리 타임라인

우선 처리 항목

우선 처리 항목이 없습니다.

화면 구성 포인트

1 핵심 상태 요약 영역 (KPI)

현재 서비스의 핵심 상태를 즉시 확인할 수 있도록 상단에 배치했습니다.  
서버 상태, 신고, 대기 상황을 카드로 시각적으로 구분했습니다.

2 운영 지표 요약 영역

기간 내 주요 운영 데이터를 카드 형태로 제공하여 운영자가 빠르게 데이터를 파악하고 이상 징후를 탐지할 수 있도록 구성했습니다.

3 신고/처리 타임라인 차트

시간 흐름에 따른 신고 및 처리 건수를 시각화하여 특정 시점의 급증/이상 패턴을 쉽게 확인할 수 있도록 했습니다.

4 우선 처리 항목

즉시 대응이 필요한 항목을 노출하는 영역으로 향후 작업 우선순위를 빠르게 파악할 수 있도록 설계했습니다.

## 실종 공고 관리

실종/발견 공고를 상태로 분류하고, 운영자가 필요한 정보를 빠르게 탐색하고 대응할 수 있도록 구성했습니다.

공고 관리

실종/발견 공고를 모니터링하고 운영합니다.

전체

실종

발견

해결

Q 제목 또는 작성자로 검색

새로고침

DESC

CSV 내보내기

리스트/그리드

랙들 찾아요

DOG 실종

부계정

2026. 5. 19.

의성군 봉양면 봉화로14

강아지를 찾습니다

DOG 실종

부계정

2026. 5. 14.

경상북도 의성군 봉양면 봉화로14

러시안 블루를 찾습니다

CAT 실종

부계:0

2026. 5. 11.

경상북도

삼색 고양이를 찾습니다.

CAT 실종

Dayum

2026. 5. 10.

서울 강남구

엑조틱 숏헤어를 찾습니다

CAT 실종

부계:0

2026. 4. 29.

전북 전주시

버먼 고양이를 찾습니다

CAT 실종

부계:0

2026. 4. 29.

충북 청주시

아비시니안을 찾습니다

CAT 실종

부계:0

2026. 4. 29.

충북 포항시

아메리칸 숏헤어를 찾습니다

CAT 실종

부계:0

2026. 4. 29.

경기 용인시

화면 구성 포인트

1 상태 필터 영역

공고 상태(전체/실종/발견/해결)를 상단에 배치하여 운영자가 필요한 상태 데이터를 빠르게 분류할 수 있도록 구성했습니다.

2 검색 및 정렬 영역

제목/작성자 검색과 정렬 기능을 제공하여 대량의 공고 중 원하는 정보를 빠르게 탐색할 수 있도록 설계했습니다.

3 공고 카드 리스트

이미지, 제목, 종류, 상태, 작성자, 날짜, 위치를 카드 단위로 그룹화하여 정보를 한눈에 확인할 수 있도록 구성했습니다.

4 상태 배지 시각화

실종/발견 상태를 색상 배지로 구분하여 긴급 대응이 필요한 공고를 빠르게 식별할 수 있도록 했습니다.

5 리스트/그리드 전환

운영 상황에 따라 리스트형과 카드형 레이아웃을 전환하여 데이터 탐색 효율성을 높였습니다.